

Script-Controlled ACL – Restrict Record Access Based on Field Value

Phase 3: Project Design

3.1 Solution Architecture

A user request to open the Institution Details list passes through the ServiceNow security layer. For each record, the platform evaluates the matching ACLs (operation = read) against the user's roles, the record's data condition and the ACL script. Only records that pass are returned.

Step	Component	Action
1	User	Opens Institution Details list (or form / portal)
2	ACL engine	Finds ACLs for table u_institution_details and operation
3	Role check	Does the user hold the required role (bb1 / bb2 / bb3 / bb4)?
4	Data condition	For read ACL: does the record have Branch = EEE?
5	Script	Admin returns true; bb1 returns true; everyone else returns false
6	Result	Record shown (all checks true) or hidden / operation blocked

3.2 Roles & ACL Design Matrix

ACL Operation	Table	Required Role	Data Condition	Advanced/Script	Effect
read	u_institution_details	bb1	Branch is EEE	Yes	View only EEE records
create	u_institution_details	bb2	None	No	New button available
write	u_institution_details	bb3	None	No	Edit records
delete	u_institution_details	bb4	None	No	Delete records

3.3 Access Decision Logic (Read ACL)

User type	Role bb1?	Branch of record	Outcome
Admin	Any	Any	Allowed (script returns true; admin overrides)
EEE User with bb1	Yes	EEE	Allowed
EEE User with bb1	Yes	ECE / CSE	Denied (data condition fails)
User without bb1	No	Any	Denied - No records to display

3.4 Script Design

```
(function () {  
  // Allow admin users full access  
  if (gs.hasRole('admin')) {  
    return true;  
  }  
  // Allow only EEE branch users to see EEE records  
  if (gs.hasRole('bb1')) {  
    return true;  
  }  
  // Deny access for all others  
  return false;  
})();
```

The script is wrapped in an immediately-invoked function and must return a boolean. The Branch restriction itself is enforced by the ACL's Data Condition (Branch is EEE); the script adds the admin bypass and the role gate.